

ATELIER C++



```
#include <iostream>
```

```
int main() {  
    std::cout << "Hello World!" << std::endl;  
    return 0;  
}
```

Notion des classes

1. Qu'est-ce qu'une classe ?

Une **classe** est un **modèle** ou un **plan** qui décrit un type d'objet.

Elle **regroupe** :

- des **données** (appelées *attributs* ou *membres de données*),
- et des **fonctions** (appelées *méthodes* ou *fonctions membres*).

👉 En résumé, **une classe = un modèle d'objet**, tandis qu'un **objet = une instance concrète** de cette classe.

Notion des classes

```
class Voiture {  
public: // zone accessible de l'extérieur  
    string marque;  
    int annee;  
  
    void demarrer() {  
        cout << "La voiture démarre " << endl;  
    }  
};
```

Nom de la classe

Visibilité des attributs et des méthodes

Attributs de la classe

Méthode de la classe

Notion des classes

- Une fois la classe définie, on peut **créer un objet** (ou instance) :

```
int main() {  
    Voiture v1;           // création d'un objet de type Voiture  
    v1.marque = "Toyota";  
    v1.annee = 2022;  
  
    cout << "Marque : " << v1.marque << endl;  
    cout << "Année : " << v1.annee << endl;  
    v1.demarrer();  
  
    return 0;  
}
```

Définition externe d'une classe

- Pour une meilleure représentation d'une classe en C++, on peut définir un prototype de la classe dans un **fichier .h** et la définition de ses méthodes dans un **autre fichier .cpp**

- **Fichier d'en-tête (.h) :**

Contient la déclaration de la classe, c'est-à-dire : les noms des attributs et des méthodes, les signatures des fonctions membres (sans leur corps). 👉 Ce fichier sert d'interface entre le programme et la classe.

- **Fichier source (.cpp) :**

Contient la définition des méthodes déclarées dans le fichier .h.

Chaque méthode est précédée du nom de la classe et de l'opérateur :: (appelé opérateur de portée).

Définition externe d'une classe

Exemple

- Le fichier **Etudiant.h**

```
#ifndef ETUDIANT_H
#define ETUDIANT_H

#include <string>
using namespace std;

class Etudiant {
private:
    string nom;
    int age;

public:
    Etudiant(string n, int a);
    void afficher();
};

#endif
```

On remarque l'utilisation des directives de compilation **#ifndef**, **#define** et **#endif** pour gérer les inclusions multiples du fichier header.

Attributs de la classe Etudiant

Méthode de la classe Etudiant

Définition externe d'une classe

Exemple

- Le fichier **Etudiant.cpp**

```
#include "Etudiant.h"
#include <iostream>
using namespace std;

Etudiant::Etudiant(string n, int a) {
    nom = n;
    age = a;
}

void Etudiant::afficher() {
    cout << "Nom : " << nom << ", Âge : " << age << endl;
}
```

Appel du Fichier **Etudiant.h**

Définition du
constructeur de la classe
Etudiant

Définition de la méthode
déclarée dans le fichier
Etudiant.h

Définition externe d'une classe

Exemple

- Le fichier **main.cpp**

```
#include "Etudiant.h"
```

Permet d'utiliser la classe Etudiant

```
int main() {  
    Etudiant e1("Amani", 25);  
    e1.afficher();  
    return 0;  
}
```

Création d'un objet e1 de type Etudiant et initialisation de ses attributs

Exécution d'une méthode sur l'objet e1

Notion des classes - This

- **En cas d'ambiguïté de nom** entre les attributs et d'autres variables dans les méthodes, on dispose du pointeur sur l'instance courante, `this`, qui nous aide à pointer les attributs de la classe.

```
void setLargeur(double largeur)
{
    this->largeur = largeur;
}
void setHauteur(double hauteur)
{
    this->hauteur = hauteur;
}
```

Constructeurs

- Les constructeurs sont des méthodes particulières qui ont pour responsabilité d'initialiser les attributs de la classe.
- Le constructeur porte le même nom que la classe, n'a pas de type de retour et est invoqué automatiquement où on crée une instance.
- Une classe peut avoir plusieurs constructeurs
- Si aucun constructeur n'est spécifié le compilateur fournit un constructeur par défaut à la classe (attention : laisse non initialisé les attributs de type de base).

Fichier Etudiant.h

```
#ifndef ETUDIANT_H
#define ETUDIANT_H

#include <string>
using namespace std;

class Etudiant
{
public:
    // Les constructeurs
    Etudiant(string n, int a);
    Etudiant();

    // Les méthodes
    string getNom() const;
    int getAge() const;
    void afficher() const;

private:
    string nom;
    int age;
};

#endif
```

Fichier Etudiant.cpp

```
#include "Etudiant.h"
#include <iostream>
using namespace std;

// constructeur avec paramètres
Etudiant::Etudiant(string n, int a)
{
    this->nom = n;
    this->age = a;
}

// constructeur par défaut
Etudiant::Etudiant()
{
    this->nom = "";
    this->age = 0;
}

// méthodes
string Etudiant::getNom() const
{
    return this->nom;
}
```


Constructeurs – Liste d'initialisation

Les listes d'initialisation servent pour initialiser les attributs et/ou d'appeler leurs constructeurs (s'ils sont des objets).

Constructeurs SANS liste d'initialisation

```
#include "Etudiant.h"

Etudiant::Etudiant(string n, int a)
{
    this->nom = n;
    this->age = a;
}

Etudiant::Etudiant()
{
    this->nom = "";
    this->age = 0;
}
```

Constructeurs AVEC liste d'initialisation

```
Etudiant::Etudiant(string n, int a)
    : nom(n), age(a)
{
}

Etudiant::Etudiant()
    : nom(""), age(0)
{
}
```

Avec valeur par défaut

```
Etudiant::Etudiant(string n = "", int a = 0)
    : nom(n), age(a)
{
}
```

Constructeurs par défaut

```
class Etudiant {  
private:  
    string nom;  
    int age;  
    // suite...  
};
```

Constructeur par défaut implicite

→ Aucun constructeur n'est défini → le compilateur génère **un constructeur par défaut implicite**, mais **les attributs ne sont pas initialisés**.

```
class Etudiant {  
private:  
    string nom;  
    int age;  
  
public:  
    Etudiant()  
        : nom(""), age(0)  
    {}  
    // suite...  
};
```

Constructeur par défaut qui initialise les attributs à 0

→ Constructeur par défaut explicite qui initialise nom à " " age à 0

Constructeurs par défaut

```
class Etudiant {  
private:  
    string nom;  
    int age;  
  
public:  
    Etudiant(string n = "", int a = 0)  
        : nom(n), age(a)  
    {}  
    // suite...  
};
```

Constructeur avec paramètres par défaut

- ➔ Ici, **un seul constructeur**, mais il joue à la fois :
- le rôle de **constructeur paramétré**,
 - et de **constructeur par défaut** (si on appelle `Etudiant e;`
→ utilise `n=""` et `a=0`).

Constructeurs par défaut

```
class Etudiant {  
private:  
    string nom;  
    int age;  
  
public:  
    Etudiant(string n, int a)  
        : nom(n), age(a)  
    {}  
    // suite...  
};
```

Pas de constructeur par défaut

- Comme il existe un constructeur paramétré **sans valeurs par défaut**,
- le compilateur **ne génère pas** de constructeur par défaut.
- Donc `Etudiant e;` n'est pas possible ❌

Exercice

- On désire représenter l'objet Point caractérisé par un abscisse et un ordonné de type réel. Un point est construit de deux manières:
 - Les coordonnées sont par défaut à 0
 - Les coordonnées sont initialisés par des valeurs utilisateurs
- Un point devrait être afficher sous cette forme (1.3 , 2.9)
- Un point peut être déplacer selon l'axe des abscisses et/ou l'axe des ordonnés.
- Définir la classe point.
- Ecrire une fonction main qui crée un point A à l'origine du repère. Puis on crée un autre point B ayant des coordonnées saisis au clavier. Afficher les deux points.
- En fin, faire confondre les deux points et les afficher de nouveau.

Exercice – Correction

```
class point
{
private:
    double abs;
    double ord;
public:
    point(double ab=0,double o=0);
    void deplacer(double, double) ;
    void afficher() const;
    double get_abs() const;
    double get_ord() const;
    void set_abs(double);
    void set_ord(double);
};
```

```
#include<iostream>
#include "point.h"

point::point(double ab, double o):abs(ab),ord(o){}
void point::deplacer(double x, double y)
{
    this->abs += x;
    this->ord += y;
}
void point::afficher() const
{
    std::cout << "(" << abs << " , " << ord << ")\n";
}
double point::get_abs() const
{
    return abs;
}
double point::get_ord() const
{
    return ord;
}
void point::set_abs(double abs)
{
    this->abs = abs;
}
void point::set_ord(double ord)
{
    this->ord = ord;
}
```


Exercice – Correction

```
#include <iostream>
#include "point.h"

int main()
{
    double x, y;
    point A;
    std::cout << "Donner les coordonnes de B:";
    std::cin >> x;
    std::cin >> y;

    point B(x,y);
    std::cout << "Le point A:";
    A.afficher();
    std::cout << "Le point B:";
    B.afficher();
    std::cout << "confondre A et B ....\n";
    A.set_abs(B.get_abs());
    A.set_ord(B.get_ord());
    std::cout << "Le point A:";
    A.afficher();
    std::cout << "Le point B:";
    B.afficher();
}
```

Constructeurs de copie

Ce genre de constructeur permet d'initialiser un objet avec un autre objet.

```
Etudiant::Etudiant(Etudiant const& e)
    : nom(e.nom), age(e.age)
{
}
```

Le compilateur crée automatiquement un constructeur de copie si le développeur n'en fournit pas. Ce constructeur de copie par défaut copie chaque attribut de l'objet, exactement comme le ferait un constructeur de copie écrit manuellement.

C'est pourquoi, dans la plupart des cas, on utilise simplement celui généré par le compilateur et on n'a pas besoin de définir un constructeur de copie nous-même.